a class hierarchy? Don't get confused—this is not a class diagram. Every box in the diagram is an object. It's just that some of these objects happen to be classes, and classes are linked together through the superclass method.

When you call my_method, Ruby goes *right* from obj, the receiver, into MySubclass. Because it can't find my_method there, Ruby continues its search by going *up* into MyClass, where it finally finds the method.

MyClass doesn't specify a superclass, so it implicitly inherits from the default superclass: Object. If it hadn't found the method in MyClass, Ruby would look for the method by climbing up the chain into Object and finally BasicObject.

Because of the way most people draw diagrams, this behavior is also called the "one step to the right, then up" rule: go *one step to the right* into the receiver's class, and then go *up* the ancestors chain until you find the method. You can ask a class for its ancestors chain with the ancestors method:

```
MySubclass.ancestors # => [MySubclass, MyClass, Object, Kernel, BasicObject]
```

"Hey, what's Kernel doing there in the ancestors chain?" you ask. "You told me about a chain of superclasses, but I'm pretty sure that Kernel is a module, not a class."

"You're right." Bill admits. "I forgot to tell you about modules. They're easy…."

Modules and Lookup

You learned that the ancestors chain goes from class to superclass. Actually, the ancestors chain also includes modules. When you include a module in a class (or even in another module), Ruby inserts the module in the ancestors chain, right above the including class itself:

object_model/modules_include.rb

```
module M1
  def my_method
    'M1#my_method()'
  end
end

class C
  include M1
end

class D < C; end

D.ancestors # => [D, C, M1, Object, Kernel, BasicObject]
```

Starting from Ruby 2.0, you also have a second way to insert a module in a class's chain of ancestors: the prepend method. It works like include, but it inserts